

GQG Core Card · v0.2

Geometric Quotient Grammar

Status: experimental language specification • **Profile:** `Set` (normative core) • **Date:** 2026-08-20 • **Attribution:** not specified in the supplied GQG source • **License:** not specified in the supplied GQG source

• **Continuity:** v0.2 distillation of the supplied GQG working material • **Purpose:** a typed visual and textual language for witness-relative quotient constructions

Claim boundary. GQG records observations, the equivalence they induce, the quotient they justify, and the proof obligations required for later operations. It does not claim that different application domains are isomorphic, that measurement is quotienting, or that a shared Hamiltonian governs those domains.

Semantic ownership

Text source → canonical AST → type checker → formal outputs

Canonical AST → glyph and diagram renderings

The abstract syntax tree owns the semantics. A rune, commutative diagram, accessible monochrome view, terminal view, and canonical JSON may all render the same accepted tree. Diagram-to-AST parsing is a future feature and must pass semantic round-trip tests before the arrow becomes bidirectional.

Canonical construction

$CAT \rightarrow SRC \rightarrow WIT \rightarrow OBS \rightarrow EQV \rightarrow QUO \rightarrow (IMG) \rightarrow \{DSC, REF, (CPL)\}$

In the normative `Set` profile:

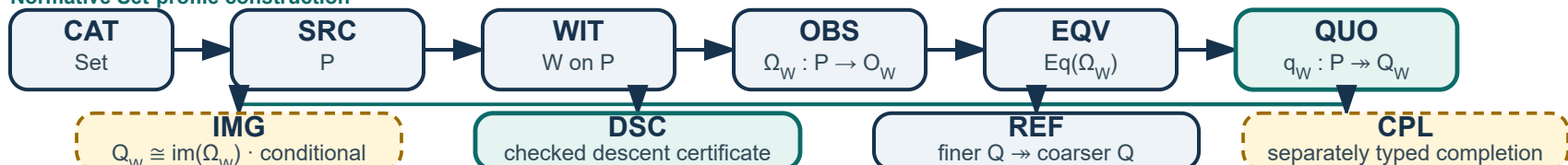
$P \rightarrow Q_W \cong \text{im}(\Omega_W) \hookrightarrow \prod_{\{w \in W\}} O_w$

Define $O_W := \prod_{\{w \in W\}} O_w$ and $\Omega_W(x) := (w(x))_{\{w \in W\}}$. Then $x \sim_W y$ exactly when $\Omega_W(x) = \Omega_W(y)$. The image comparison is conditional in other backends: the profile must declare the needed products, kernel pairs, coequalizers, images, and exactness assumptions.

Token alphabet

Token	Typed role
CAT	Ambient category or structural profile
SRC	Presented source object P
WIT	Witness family W on P
OBS	Aggregate observation $\Omega_W : P \rightarrow O_W$
EQV	Kernel pair / induced equivalence $\sim_W$
QUO	Quotient object and map $q_W : P \twoheadrightarrow Q_W$
IMG	Backend-supported image factorization
DSC	Certificate that an operation descends
REF	Witness refinement and quotient comparison
CPL	Separately typed completion specification

Normative `Set`-profile construction



Equality lanes

Lane	Form	Meaning
Source	$x = y$	The presented elements are equal.
Observation	$x \sim_W y$	The declared witnesses cannot distinguish them.
Quotient	$[x]_W = [y]_W$	They name the same quotient class.
Completion	$u = v$ in $\hat{Q}_W$	Equality may involve new completion elements; the declared embedding must state what it preserves and reflects.

Never move a claim between lanes without an explicit constructor or proof.

Formation rules

1. Declare `CAT` before `SRC`; v0.2 accepts only the explicit declaration `category Set`.
2. Every `WIT` names its source and every `OBS` has a complete domain and codomain.
3. For an arbitrary map, `EQV` means the kernel pair—not an algebraic kernel.
4. Form `QUO` only when the active profile supports the required quotient.
5. Emit `IMG` and $Q_W \cong \text{im}(\Omega_W)$ only when the backend justifies that factorization.
6. Defining an operation on Q_W induced by a source operation requires `DSC`: if each $x_i \sim_W y_i$, then $f(x_1, \dots, x_n) \sim_W f(y_1, \dots, y_n)$. The certificate must reference a checked proof term or an externally verified obligation.
7. A finer witness system induces a finer quotient, and that finer quotient maps canonically onto the coarser quotient; the quotient arrow points fine → coarse.
8. `CPL` must name its added structure (for example, metric, order, topology, or norm). `Set` alone cannot justify a Dedekind completion.

Worked card

Minimal source form

```
category Set
source P = Integers
codomain ParityBits = Z/2Z
witness Parity on P {
  observation omega : P -> ParityBits
  definition omega(n) = n mod 2
}
equivalence same_parity := kernel_pair(omega)
quotient Q2 := P / same_parity
operation add : P × P -> P
certificate add_respects_parity {
  require n ~2 n' and m ~2 m'
  prove n + m ~2 n' + m'
}
descend add through Q2 using add_respects_parity
```

Parity example

Let $P = \mathbb{Z}$ and $\Omega_2(n) = n \bmod 2$. Then $n \sim_2 m \Leftrightarrow \Omega_2(n) = \Omega_2(m) \Leftrightarrow n - m \in 2\mathbb{Z}$, so $Q_2 \cong \mathbb{Z}/2\mathbb{Z}$. Addition passes descent: $n \sim_2 n'$ and $m \sim_2 m' \Rightarrow n + m \sim_2 n' + m'$. The operation $h(n) = \text{floor}(n/2)$ fails. Although $0 \sim_2 2$, its proposed quotient outputs differ: $[h(0)]_2 = [0]_2 \neq [1]_2 = [h(2)]_2$. The checker therefore accepts `DSC[add, add_respects_parity]` after checking its certificate and rejects `DSC[h]`.

Witness refinement

For $\Omega_4(n) = n \bmod 4$ and $\Omega_2(n) = n \bmod 2$, the mod-4 witness is finer: $\sim_4 \subseteq \sim_2$, hence $Q_4 \twoheadrightarrow Q_2$. Combining witness families intersects their induced equivalence relations and produces a common refinement in the `Set` profile.

Diagnostic catalogue

Code	Rejection
E101	QUO has no declared EQV.
E204	An induced quotient operation has no valid DSC certificate.
E301	CPL omits the structure and universal property being added.
E407	A refinement arrow points coarse $\rightarrow$ fine.
E512	The requested quotient–image comparison is unsupported by the active profile.
E610	A statement crosses equality lanes without a constructor or proof.

Illustrative typed machine record

```
{
  "schema": "gqg-set-0.2",
  "profile": "Set",
  "objects": {
    "P": {"kind": "Set", "presentation": "Integers"},
    "ParityBits": {"kind": "Set", "presentation": "Z/2Z"},
    "Q2": {"kind": "QuotientSet"}
  },
  "witness": "Parity",
  "observation": {
    "name": "omega",
    "domain": "P",
    "codomain": "ParityBits",
    "definition": "n mod 2"
  },
  "equivalence": "kernel_pair(observation)",
  "quotient": {
    "id": "Q2",
    "map": {"name": "q2", "domain": "P", "codomain": "Q2"}
  },
  "descent": {
    "operation": "add",
    "certificate": "add_respects_parity",
    "status": "checked"
  }
}
```

This compact record is illustrative: a production schema must replace free-form expressions with expression AST nodes, define stable identifiers, and specify a JSON canonicalization algorithm. Only then should canonical identity be a digest of canonical JSON for the AST. Prime products may remain an unordered feature signature—useful for indexing which token types occur—but they are not syntax because they lose order, direction, scope, and binding.

v0.2 acceptance target

- **Parseability:** one valid sentence produces one unambiguous AST.
- **Soundness:** accepted trees satisfy the declared formation rules.
- **Rendering stability:** text and diagrams emitted from one accepted AST preserve the same semantic structure. Any future diagram parser must pass round-trip tests before use.
- **Diagnostic value:** rejected trees receive local, actionable errors.
- **Accessible rendering:** meaning survives without color or decorative glyphs.

Implementation milestone: parse the `Set` profile, validate the parity example, emit canonical JSON, and render both a labelled diagram and an accessibility-first text view from the same AST.